

栈与队列

1 给定一个栈S，保存的数据类型均为int，其仅具有如下三个运算：

S.push(x)：将x压入栈中

S.pop()：将栈顶元素出栈，并返回该元素

S.top()：返回栈顶元素

请你在该栈的基础上，不添加新的数据结构，使得S可以仍然支持上述三种运算（可以重写），并实现一

种新的运算：

S.getMin()：返回栈中最小元素

要求：S的所有操作时间复杂度为 $O(1)$ 。无需考虑特殊情况如栈空或者栈满等。请给出所有新运算的伪代码。

可以通过重写数据结构S，使得其保存的每个元素都含有当前的栈内最小元素，存储元组 (value, current_min)

```
global currentMin = 1e9//或者支持正无穷的话
```

```
// 重写push
```

```
procedure push(x)
    if x < currentMin then
        currentMin = x
    end if
    // 将元素和当前最小值一起压入栈
    S.push((x, currentMin))
end procedure
```

```
// 重写pop
```

```
function pop()
    if S is empty then
        return null
    end if

    (value, minVal) = S.pop()

    // 如果栈不为空，更新currentMin为新的栈顶的最小值
    if not S.isEmpty() then
        (_, newMin) = S.top()
        currentMin = newMin
    else
        currentMin =  $\infty$ 
    end if

    return value
end function
```

```
// 重写top
```

```
function top()
    if S is empty then
        return null
    end if

    (value, _) = S.top()
    return value
end function
```

```
// 实现getMin
```

```
function getMin()
    if S is empty then
```

```
        return null
    end if

    return currentMin
end function
```

2. 编号为 1, 2, ..., n 的 n 辆火车顺序开进栈式结构的站台。请问开出车站的顺序有多少种可能？请写出你的推导过程。

设

$f(n)$ 表示 n 辆火车的出栈序列数量。考虑第一辆开出车站的火车编号为 k (其中 $1 \leq k \leq n$)。根据栈的性质 (后进先出)，在火车 k 开出之前，火车 1 到 k-1 必须已经进入站台并开出，且它们的出栈顺序必须遵循栈规则，因此这部分有 $f(k-1)$ 。在火车 k 开出后，剩下的火车 k+1 到 n 必须进入站台并开出，这部分有 $f(n-k)$ 种可能。

又 k 可以从 1 到 n 任意取值，所以 $f(n) = \sum_{k=1}^n f(k-1)f(n-k)$ ，因为符合卡特兰数的定义，则

$$f(n) = C_n = \frac{1}{n+1} \binom{2n}{n}$$

3.

在文本编辑器中，有一个“撤销 (Undo)”和“恢复 (Redo)”的功能。假设编辑器使用两个栈来实现：栈 S1 保存已经执行的操作 (每次新操作都会压入 S1)；

栈 S2 保存被撤销的操作 (每次执行一次撤销，就把 S1 的栈顶弹出并压入 S2；每次执行一次恢复，就把 S2 的栈顶弹出并压入 S1)。

现在给定一系列操作指令 (操作包括：

do x：执行一个新操作 x；

undo：撤销一步操作；

redo：恢复一步操作)，

请回答：

1. 用栈的基本操作（push、pop、empty）描述撤销和恢复的实现过程。

```
procedure UNDO()  
    if not S1.empty() then  
        operation = S1.pop()  
        S2.push(operation)      // 压入撤销栈  
    end if  
end procedure  
  
procedure REDO()  
    if not S2.empty() then  
        operation = S2.pop()    // 从撤销栈弹出顶部操作  
        S1.push(operation)      // 压入已执行栈  
    end if  
end procedure  
  
procedure DO(operation)  
    S1.push(operation)  
end procedure
```

2. 如果一开始编辑器为空，依次执行以下操作序列：

do A, do B, do C, undo, do D, undo, redo

请问最后 S1 和 S2 中分别保存哪些操作？

执行过程如下：

执行指令	描述	栈内
do A	将 A 压入 S1	S1: [A], S2: []
do B	将 B 压入 S1	S1: [A, B], S2: []
do C	将 C 压入 S1	S1: [A, B, C], S2: []
undo	从 S1 弹出 C 并压入 S2	S1: [A, B], S2: [C]
do D	将 D 压入 S1	S1: [A, B, D], S2: [C]
undo	从 S1 弹出 D 并压入 S2	S1: [A, B], S2: [C, D]
redo	从 S2 弹出 D 并压入 S1	S1: [A, B, D], S2: [C]

最终状态

S1 中从栈底到栈顶：A, B, D

S2 中从栈底到栈顶的操作：C

4已知队列的三个基本操作定义如下：

enqueue(Q, x)：将元素 x 入队列 Q（在队尾插入）。

dequeue(Q)：队列 Q 的队首元素出队，并返回该元素。

isEmpty(Q)：判断队列 Q 是否为空。

请你设计一种方法，用两个普通队列（Q1, Q2）来实现栈（Stack）的三个基本操作：

push(x)：将元素压入栈顶。

pop()：弹出并返回栈顶元素。

isEmpty()：判断栈是否为空。

1. 给出算法思路，并写出 push 和 pop的伪代码

```
// push操作
procedure PUSH(x)
    if not isEmpty(Q1) then
        enqueue(Q1, x)
    else
        enqueue(Q2, x)
    end if
end procedure

// pop操作
function POP()
    if isEmpty(Q1) and isEmpty(Q2) then
        return error // 栈为空
    end if

    // 确定哪个队列非空
    if not isEmpty(Q1) then
        // 将Q1中除最后一个元素外的所有元素转移到Q2
        while size(Q1) > 1 do
            enqueue(Q2, dequeue(Q1))
        end while
        // 返回最后一个元素
        return dequeue(Q1)
    else
        // 将Q2中除最后一个元素外的所有元素转移到Q1
        while size(Q2) > 1 do
            enqueue(Q1, dequeue(Q2))
        end while
        // 返回最后一个元素
        return dequeue(Q2)
    end if
end function

// 判断栈是否为空
function IS_EMPTY()
    return isEmpty(Q1) and isEmpty(Q2)
end function
```

2. 分析 push 和 pop 的操作的时间复杂度。

- push操作： $O(1)$ 因为是直接将元素加入队列，不涉及元素转移
- pop操作： $O(n)$ ，因为需要将 $n-1$ 个元素从一个队列转移到另一个队列，而每个元素都需要执行一次出队和入队操作

3. 若依次执行以下操作序列：

push(1), push(2), push(3), pop(), push(4), pop(), pop()

请写出每次pop() 的返回结果

- 第一次pop：3
- 第二次pop：4
- 第三次pop：2